Juan Antonio Robledo Menéndez
Javier García Aguilar
Gonzalo Gómez Gracia
Práctica 2, PKI, Laboratorio ESI
31 Octubre 2025

Para la realización de la práctica hemos acordado los siguientes valores comunes a la hora de generar los certificados:

- Código país: ES
- Nombre de estado: Andalucia (sin tildes)
- Provincia: Sevilla
- Organización: Universidad de Sevilla
- Departamento: DIT

1. Generación del certificado de CA e infraestructura asociada

Primero generamos una copia del fichero de configuración de la suite OpenSSL a la que apuntaremos con la opción `-config` de la herramienta cuando realicemos el resto de operaciones para que tanto los certificados firmados como el servidor OCSP utilicen la información acordada en la práctica y los ficheros necesarios (como la base de datos del servidor OCSP `inputs.txt`). El extracto de la configuración de OpenSSL que hemos modificado es el siguiente:

```
# Fichero: /home/dit/certs/openssl/openssl.cnf
[ CA_default ]

dir           = /home/dit/certs/CA     # Directorio base
certs         = $dir/certs        # Localizacion de los certificados
crl_dir       = $dir/ocsp/crl     # Usamos un directorio 'ocsp' para separar
                                  # la informacion relativa a este servicio
                                  # del resto de operaciones de la CA
database      = $dir/ocsp/index.txt
new_certs_dir = $dir/newcerts
certificate   = $dir/CAcert.pem # Certificado de la CA
serial        = $dir/ocsp/serial
crlnumber     = $dir/ocsp/crlnumber
crl           = $dir/ocsp/crl/crl.pem     # Certificado utilizado por el servidor OCSP
private_key   = $dir/private/CAkey_crypt.pem # Clave privada (encriptada)
```

Además, en la sección `req_distinguished_name` del fichero de configuración de OpenSSL configuramos los valores por defecto para los certificados (código país, organización, etc) acordados al inicio de la práctica. Adicionalmente, configuramos el parámetro authorityInfoAccess con la IP y el puerto (usamos uno de los denominados “puertos altos” de Linux, superior al 1024, para poder ejecutar el servidor OCSP sin privilegios de administración, por ejemplo el puerto 8123) en el que el servidor OCSP estará escuchando:

```
dit@maquina_CA: $ ip add l
(...)
inet 172.16.17.224/24

# fichero /home/dit/certs/openssl/openssl.cnf
(...)
authorityInfoAccess = OCSP;URI:http://172.16.17.224:8123
```

Este último parámetro irá incluido en uno de los campos de información extendida que incluirán los certificados firmados por esta CA, para que los clientes finales sepan dónde preguntar por el estado de un certificado (¿válido o revocado?).

Haciendo uso de los comandos `mkdir` y `touch` generamos la estructura del sistema de archivos necesario para el funcionamiento de la CA (directorios de certificados, OCSP, base de datos de OCSP, etc.). Por último, y al igual que los usuarios finales, la CA genera su propio par de claves asimétricas con el algoritmo AES 256 y luego lo encripta con AES-256-CBC para obtener su clave cifrada (con una contraseña que sólo el gestor de la CA conoce), y que será la que utilice para el resto de acciones (firmar las solicitudes que reciba y revocar los certificados que sean necesarios):

```
$ openssl genrsa -out CAkey.key 2048
$ openssl enc -aes-256-cbc -salt -in CAkey.key -out CAkey_crypt.pem
```

Con esta información, la CA puede generar su propio certificado autofirmado en el que el resto de usuarios confiarán. También podemos inspeccionar el certificado usando el comando openssl:

```
$ cd /home/dit/certs/CA
$ openssl req -new -x509 -key ./private/CAkey_crypt.pem -out ./CAcert.pem -config
../openssl/openssl.cnf
$ openssl x509 -noout -text -in CAcert.pem
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            3b:1e:72:8f:11:d4:b4:0d:67:c7:e3:e9:c8:8a:db:b4:cb:3e:6a:4e
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = ES, ST = Andalucia, L = Sevilla, O = Universidad de Sevilla, OU =
DIT, CN = Gonzalo Gomez CA, emailAddress = gracia@trajano.us.es
        Validity
            Not Before: Oct 31 16:52:51 2025 GMT
            Not After : Nov 30 16:52:51 2025 GMT
        Subject: C = ES, ST = Andalucia, L = Sevilla, O = Universidad de Sevilla, OU =
DIT, CN = Gonzalo Gomez CA, emailAddress = gracia@trajano.us.es
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                  (...)
```

```
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                1E:C7:09:52:99:93:C5:2B:8F:F4:29:75:E6:6D:93:FA:E2:11:EE:42
            X509v3 Authority Key Identifier:
                1E:C7:09:52:99:93:C5:2B:8F:F4:29:75:E6:6D:93:FA:E2:11:EE:42
            X509v3 Basic Constraints:
                CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
      (...)
```

Como podemos ver en la salida anterior, la propia autoridad certificadora *Gonzalo Gomez CA* certifica al sujeto *Gonzalo Gomez CA* (es decir, autofirmado). Este certificado (`CA_cert.pem`) es compartido por la CA con los usuarios finales, que lo marcarán como de confianza en sus aplicaciones de correo electrónico (ver punto 4).

2. Generación de la solicitud de certificación de los usuarios A y B

Por simplicidad sólo se detallan los pasos de uno de los usuarios

Primero generamos un par de claves mediante el algoritmo RSA, ya que, a diferencia de otros como DSA, nos permite firmar y cifrar. Posteriormente, el fichero con el par de claves generadas se encripta para restringir su acceso y garantizar que solamente el propietario pueda hacer uso de ellas.

```
$ openssl genrsa -aes256 -out javi_crypt.pem 2048
Enter PEM pass phrase: <valor secreto>
Verifying - Enter PEM pass phrase: <valor secreto>

$ ls
javi_crypt.pem
```

Si comparamos el contenido de un fichero que almacena las claves cifradas con uno que no lo hace, podemos ver a simple vista como las cabeceras difieren. En el caso de la no cifrada, la primera línea comienza por '`-----BEGIN PRIVATE KEY-----`' seguida de caracteres codificados en base64, mientras que en la contraria aparece '`-----BEGIN ENCRYPTED PRIVATE KEY-----`'. Ambos se encuentran en formato `PKCS#8`.

Usando dicha clave, generamos la solicitud de certificado con extensión `.csr`:

```
$ openssl req -new -key javi_crypt.pem -out javi_cert.csr
Enter pass phrase for javi_crypt.pem: <valor secreto>
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
```

```
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Andalucia
Locality Name (eg, city) []:Sevilla
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Universidad de Sevilla
Organizational Unit Name (eg, section) []:DIT
Common Name (e.g. server FQDN or YOUR name) []:Javier García Aguilar
Email Address []:aguilar@trajano.us.es

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:password
An optional company name []:Universidad de Sevilla
```

Y enviamos la solicitud de firma (fichero `javi_cert.csr`) a la autoridad certificadora por email (en este caso a la dirección `gracia@trajano.us.es`). Esperamos su respuesta con el certificado firmado.

3. Firma de certificados por parte de la CA

Mediante el comando `openssl ca`, y el fichero de configuración personalizado de OpenSSL, que contiene las rutas a la clave privada de la CA y la dirección del servicio OCSP entre otros, la autoridad certificadora puede firmar las solicitudes ejecutando el siguiente comando:

```
$ openssl ca -config ../../openssl/openssl.cnf -in javi_cert.csr -out javi_cert.crt
-verbose
Using configuration from ../../openssl/openssl.cnf
Enter pass phrase for /home/dit/certs/CA/private/CAkey_crypt.pem:
0 entries loaded from the database
generating index
message digest is sha256
policy is policy_match
next serial number is 01
Certificate Request:
    Data:
        Version: 1 (0x0)
        Subject:
            countryName               = ES
            stateOrProvinceName       = Andalucia
            localityName              = Sevilla
            organizationName          = Universidad de Sevilla
            organizationalUnitName    = DIT
            commonName                = Javier Garc\C3\ADa Aguilar
            emailAddress              = aguilar@trajano.us.es
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:c0:e4:e2:29:a4:78:92:37:01:a8:ee:13:91:8d:
                    55:32:5f:84:0d:f9:ea:f0:77:ae:e5:b7:b3:61:34:
                    e5:1b:31:6e:9d:79:f4:44:17:57:76:0a:9e:18:a3:
                    49:7a:fd:1f:0a:b1:57:d0:3c:a9:81:3e:47:5e:e5:
                    ef:4b:51:b4:21:45:a4:a6:12:ce:67:f1:ac:79:fc:
```

```
                16:03:78:5a:72:14:7d:0e:84:6f:74:64:6c:a8:d3:
                c4:37:a9:34:e2:82:84:7a:06:3b:84:28:8f:b8:18:
                3d:13:69:ed:c4:b6:50:4d:16:66:19:c0:81:1b:4b:
                55:9d:c4:6c:c5:37:0e:8b:e9:f3:3b:6e:78:c5:79:
                b5:1a:81:a9:96:df:c1:f2:25:c2:dd:2c:a3:4d:b5:
                b4:1e:66:3d:3f:3f:5c:9e:9d:55:27:b7:d3:6b:92:
                01:37:25:69:e6:eb:50:2e:be:7e:c0:ad:7f:3d:ac:
                10:dd:3c:06:58:79:9c:d6:85:6e:12:ad:d1:bb:a0:
                0d:10:d6:00:8a:11:7b:5e:95:82:9d:03:24:40:35:
                d0:6e:4b:ee:b7:b5:1d:b2:0a:f9:19:ea:90:f8:5a:
                ca:18:8d:dc:11:9d:05:b1:1c:5d:e9:36:ed:f9:6b:
                fe:9b:bb:ab:33:7a:78:6b:e3:fd:8c:e8:ae:5b:94:
                41:f7
            Exponent: 65537 (0x10001)
        Attributes:
            challengePassword        :password
            unstructuredName         :Universidad de Sevilla
            Requested Extensions:
            Modulus:
                00:c0:e4:e2:29:a4:78:92:37:01:a8:ee:13:91:8d:
                55:32:5f:84:0d:f9:ea:f0:77:ae:e5:b7:b3:61:34:
                e5:1b:31:6e:9d:79:f4:44:17:57:76:0a:9e:18:a3:
                49:7a:fd:1f:0a:b1:57:d0:3c:a9:81:3e:47:5e:e5:
                ef:4b:51:b4:21:45:a4:a6:12:ce:67:f1:ac:79:fc:
                16:03:78:5a:72:14:7d:0e:84:6f:74:64:6c:a8:d3:
                c4:37:a9:34:e2:82:84:7a:06:3b:84:28:8f:b8:18:
                3d:13:69:ed:c4:b6:50:4d:16:66:19:c0:81:1b:4b:
                55:9d:c4:6c:c5:37:0e:8b:e9:f3:3b:6e:78:c5:79:
                b5:1a:81:a9:96:df:c1:f2:25:c2:dd:2c:a3:4d:b5:
                b4:1e:66:3d:3f:3f:5c:9e:9d:55:27:b7:d3:6b:92:
                01:37:25:69:e6:eb:50:2e:be:7e:c0:ad:7f:3d:ac:
                10:dd:3c:06:58:79:9c:d6:85:6e:12:ad:d1:bb:a0:
                0d:10:d6:00:8a:11:7b:5e:95:82:9d:03:24:40:35:
                d0:6e:4b:ee:b7:b5:1d:b2:0a:f9:19:ea:90:f8:5a:
                ca:18:8d:dc:11:9d:05:b1:1c:5d:e9:36:ed:f9:6b:
                fe:9b:bb:ab:33:7a:78:6b:e3:fd:8c:e8:ae:5b:94:
                41:f7
            Exponent: 65537 (0x10001)
        Attributes:
            challengePassword        :password
            unstructuredName         :Universidad de Sevilla
            Requested Extensions:
Check that the request matches the signature
Signature ok
Everything appears to be ok, creating and signing the certificate
Successfully added extensions from config
The subject name appears to be ok, checking database for clashes
Certificate Details:
        Serial Number: 1 (0x1)
        Validity
            Not Before: Oct 31 17:19:40 2025 GMT
            Not After : Oct 31 17:19:40 2026 GMT
        Subject:
            countryName               = ES
            stateOrProvinceName       = Andalucia
```

```
            organizationName          = Universidad de Sevilla
            organizationalUnitName    = DIT
            commonName                = Javier Garc\C3\ADa Aguilar
            emailAddress              = aguilar@trajano.us.es
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            Authority Information Access:
                OCSP - URI:http://172.16.17.224:8123
            Netscape Cert Type:
                SSL Client, S/MIME, Object Signing
            Netscape Comment:
                OpenSSL Generated Certificate
            X509v3 Subject Key Identifier:
                65:3A:17:21:4B:3D:C4:C1:F2:76:B1:A9:03:A5:20:A0:83:D6:95:38
            X509v3 Authority Key Identifier:
                1E:C7:09:52:99:93:C5:2B:8F:F4:29:75:E6:6D:93:FA:E2:11:EE:42
            Netscape CA Policy Url:
                http://trajano.us.es/ca/policy.pdf
Certificate is to be certified until Oct 31 17:19:40 2026 GMT (365 days)
Sign the certificate? [y/n]:y
1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
writing new certificates
writing /home/dit/certs/CA/newcerts/01.pem
Database updated
```

Como puede apreciarse (y por facilidad de lectura ha sido destacado en negrita en la salida anterior), se ha utilizado la clave privada de la CA (`CAkey_crypt.pem`) para generar el certificado con número de `serie 01` para el sujeto `Javier García Aguilar` con dirección de email `aguilar@trajano.us.es` con una validez de `365 días`. También podemos confirmar que el nuevo certificado incluye la dirección del servidor OCSP configurada al principio de la práctica y que está definida en el fichero de configuración `openssl.cnf`.

Además, podemos confirmar que el certificado de la CA va incluido en el fichero generado:

```
$ openssl x509 -noout -text -in javi_cert.crt
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 1 (0x1)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = ES, ST = Andalucia, L = Sevilla, O = Universidad de Sevilla, OU =
DIT, CN = Gonzalo Gomez CA, emailAddress = gracia@trajano.us.es
        Validity
            Not Before: Oct 31 17:19:40 2025 GMT
            Not After : Oct 31 17:19:40 2026 GMT
        Subject: C = ES, ST = Andalucia, O = Universidad de Sevilla, OU = DIT, CN =
Javier Garc\C3\83\C2\ADa Aguilar, emailAddress = aguilar@trajano.us.es
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
```

```
(...)
```

De manera similar, la CA firma el certificado del otro usuario con el número de `serie 02`, y por último comparte los ficheros generados con los usuarios finales mediante correo electrónico.

4. Importar y marcar el certificado de la CA como de confianza

Desde la interfaz gráfica de Thunderbird, navegamos al menú `Ajustes > Privacidad y Seguridad > Administrar Certificados`. Desde esta sección, importamos el certificado de la CA que nos ha sido proporcionado por la misma y lo marcamos como de confianza en el apartado de `autoridades`.

5. Instalación del certificado de usuario y clave privada en Thunderbird

Antes de instalar nuestro certificado en el cliente de correo, comprobamos que la clave pública incluida en el certificado que la CA nos ha enviado coincide con la que está incluida en el fichero que generamos en el punto 2:

```
$ openssl x509 -in javi_cert.crt -noout -modulus > modulus_crt_ca
$ openssl rsa -in javi_crypt.pem -noout -modulus > modulus_private
$ diff modulus_crt_ca modulus_private
$
```

El comando diff no devuelve ninguna diferencia entre ambas claves públicas, luego podemos confirmar que el certificado que nos ha enviado la CA (más allá del nombre del fichero, que podría tener un valor aleatorio), es el que nos corresponde como sujetos.

A continuación generamos un fichero de almacenamiento en formato `PKCS#12`. Este formato estándar, permite incluir en un mismo fichero cifrado las claves privadas y públicas junto con los certificados correspondientes, y puede ser compartido con otras utilidades (en nuestro caso con el gestor de correo electrónico Thunderbird) para que dichas claves sean utilizadas por esos programas. Generamos el fichero `PKCS#12` con la utilidad openssl con el siguiente comando:

```
$ openssl pkcs12 -export -in javi_cert.crt -inkey javi_crypt.pem -out javi_cert.p12
Enter pass phrase for javi_crypt.pem:
Enter Export Password: <valor secreto>
Verifying - Enter Export Password: <valor secreto>
```

Para instalarlo en Thunderbird, entramos en `Ajustes > Privacidad y Seguridad > Administrar Certificados`, e importamos el fichero `PKCS#12` en el apartado de `sus certificados` utilizando la clave de exportación definida anteriormente (`<valor secreto>`).

Una vez instaladas nuestras claves en el gestor de correo, configuramos su uso desde el menú `Configuración de la Cuenta > Cifrado extremo a extremo > S/MIME`. En este menú, configuramos nuestro certificado para la firma y el cifrado de email.

Así mismo, importamos el certificado del otro usuario para poder cifrar los mensajes de correo electrónico que sólo su clave privada pueda desencriptar. En nuestro caso, y por comodidad, dejamos que Thunderbird descifre los mensajes usando la clave privada que le hemos instalado con el fichero en formato `PKCS#12`.

6. Envío de mensajes firmados y/o cifrados entre los usuarios A y B

Llegados a este punto, ambos usuarios pueden enviarse mensajes firmados digitalmente (y certificados por la entidad de confianza `Gonzalo Gomez CA`) y/o cifrados desde el menú `S/MIME` del editor de correo electrónico durante la escritura del mismo.

En el otro extremo encontramos un icono con la *forma de un sello* y el texto `S/MIME` al abrir el correo electrónico recibido que nos confirma la validez de la firma incluida en el mensaje. También es posible inspeccionar el texto cifrado desde el menú `Ver > Formato original del mensaje`.

7. Ejecución del servicio OCSP

Para ejecutar el servidor OCSP, la CA genera un nuevo certificado y lo autofirma

```
$ openssl req -config ./openssl.cnf -new -newkey rsa:2048 -nodes -out ocsp.csr -keyout
ocsp.key -extensions v3_OCSP
$ openssl ca -config ./openssl.cnf  -extensions v3_OCSP -in ocsp.csr -out ocsp.crt
```

Con estos elementos levantamos el servicio OCSP desde otra terminal:

```
$ openssl ocsp -index ./index.txt -port 8123 -rsigner ../ocsp/ocsp.crt -rkey
../ocsp/ocsp.key -CA ./CAcert.pem -text -out ./log.txt -ignore_err
```

> El argumento -ignore_err nos ayudará a mantener levantado el servicio en caso de un error interno o recibir una consulta errónea.

8. Revocación de certificados por parte de la CA

Para verificar el uso del servicio `OCSP` desde uno de los clientes, la autoridad certificadora procede a la revocación del certificado con número de `serie 02` aludiendo que la clave ha sido comprometida, y generamos la lista de revocación:

```
$ openssl ca -config ../openssl/openssl.cnf -revoke newcerts/02.pem -crl_reason
keyCompromise
$ openssl ca -config ./openssl/openssl.cnf -gencrl -out CA/crl/crl.pem
```

Para obtener información adicional de las consultas, y verificar el contenido de los paquetes intercambiados entre el cliente de correo y el servidor OCSP, hemos ejecutado el programa Wireshark en el servidor y hemos filtrado por el protocolo OCSP.

Por último, el usuario que ha visto su certificado revocado (*B*, con número de `serie 02`), envía un nuevo mensaje al usuario A (número de `serie 01`). En el momento de abrir el mensaje recibido, el cliente hace una consulta al servidor OCSP para confirmar el estado del certificado con número de `serie 02`, y que es capturada por Wireshark. En dicha consulta, dentro de la sección `Online Certificate Status Protocol > tbsRequest > request list > Request > reqCert > hash Algorithm (SHA-1)` confirmamos que se ha solicitado la información del certificado con `número de serie 02` mediante el atributo `serialNumber: 0x02`.

Del mismo modo, en la respuesta también capturada por Wireshark podemos comprobar que el certificado ha sido revocado con el motivo `keyCompromise`. Esta información se traduce en el gestor de correo Thunderbird con un icono en color rojo al lado del texto `S/MIME`. Haciendo click en este botón, el gestor nos confirma que la firma incluida en el mensaje recibido *no ha podido ser verificada* porque el certificado está revocado por la entidad certificadora.